DistroForge

Manual técnico · versión 1.0

App web, app de escritorio y backend
17/10/2026

Constructor de distribuciones Linux: de una receta a una ISO, una imagen de disco, un contenedor OCI o una imagen WSL.

1. Qué es DistroForge

DistroForge monta distribuciones Linux personalizadas desde el navegador. Eliges la base, el escritorio, los paquetes, los repositorios, el sistema y la identidad visual, y la herramienta genera el proyecto de construcción completo: la receta del motor que elijas, un build.sh para lanzarlo en local y un workflow de GitHub Actions para construirlo en CI. El resultado se descarga como zip y, si quieres, se comparte por enlace o se publica en la galería comunitaria.

Es el sucesor espiritual de SUSE Studio, que se apagó en 2018. De aquel modelo conserva lo que funcionaba: navegación por pestañas no lineal, un resumen persistente de la receta, catálogo por grupos con recuento y una vista previa del escritorio. El Test Drive —arrancar la imagen en el navegador sin descargarla— también viene de ahí.

Importante: la ISO no se compila dentro del navegador. Lo que se genera es el proyecto de construcción (recetas, scripts y CI) que produce la imagen allí donde lo lances.

2. Arquitectura

Todo el código es Rust desde la v0.4.0: la UI web (Leptos/WASM), el núcleo generador, el CLI, el backend (axum) y la app de escritorio (Tauri 2).

Componente	Tecnología	Responsabilidad
forge-core	Rust (crate)	Catálogo, validación y generadores de recetas. ~80 tests.
forge-cli	Rust (binario `forge`)	Vuelca a disco el proyecto generado sin interfaz.
forge-wasm	wasm-bindgen	Expone el núcleo a la app web.
web-ui	Leptos 0.7 CSR + Trunk	La app: seis pasos, vista previa, galería y sección Servidor.
server-rs	axum + rusqlite	Cuentas, galería con FTS5 y Test

		Drive VNC.
tauri-app	Tauri 2	App de escritorio que sirve el bundle de la UI.

La UI es CSR (client-side rendering): se compila a WebAssembly y todo el trabajo de generación ocurre en el navegador. El backend es opcional y solo aporta lo que necesita servidor: cuentas, galería pública compartida y Test Drive.

3. El catálogo

3.1 Bases

Familia	Versiones	Gestor	Motor por defecto
openSUSE	Tumbleweed, Leap 16, Slowroll, MicroOS	zypper	KIWI NG
Debian	Debian 13, Ubuntu 24.04, Mint, elementary, Pop!_OS, Zorin	apt	live-build
Arch	Arch, Manjaro, CachyOS	pacman	mkarchiso
Fedora	Fedora 44	dnf	kickstart + livemedia-creator
Gentoo	rolling	portage	guía manual
Solus	rolling	eopkg	guía manual

3.2 Motores

Motor	Para qué base	Qué produce
KIWI NG	openSUSE / SUSE	config.xml + config.sh + árbol de overlay; ISO, QCOW2, OCI
live-build	Debian y derivados	config/ completo con preseed; ISO híbrida y USB
mkarchiso	Arch y derivados	perfil de archiso (profiledef.sh, packages.x86_64)
kickstart	Fedora y derivados	kickstart + livemedia-creator; ISO y disco

3.3 Salidas y arquitecturas

Formatos: ISO híbrida, QCOW2, USB/raw, VMDK, VDI, VHD, OCI (con SBOM syft, escaneo trivy y firma cosign opcional) y WSL. Arquitecturas: x86_64, aarch64, armhf (con QEMU) e i386. El CI del proyecto generado usa runners ARM nativos cuando la arquitectura es aarch64.

4. Los seis pasos

La navegación es por pestañas y no lineal: puedes completar los pasos en cualquier orden. El resumen de la izquierda muestra en todo momento el estado de la receta, el recuento de paquetes y el espacio estimado.

Paso	Qué se decide	Consecuencias
Base	Familia, versión, plantilla y arquitectura	Fija el motor, los repos y los valores por defecto
Paquetes	Paquetes sueltos, grupos y packs de la galería	Afecta al tamaño y a las dependencias resueltas
Repos	Repositorios extra y sus claves GPG	Los ficheros de repo se firman en la receta
Sistema	Usuarios, servicios, red, cortafuegos, locale, GRUB	Se traduce a cada motor (kickstart, preseed, AutoYaST)
Identidad	Logo, fondo, MOTD, EULA, auto-login, icono de inicio	Vista previa WYSIWYG de GRUB, escritorio y MOTD
Salida	Formato, arquitectura, compresión y extras	Define los artefactos del build y del CI

Además: compartir por enlace (la receta viaja codificada en la URL), exportar e importar distroforge.json, plantillas rápidas, modpacks aditivos y estimación de tamaño en vivo.

5. Galería comunitaria

Los packs viven en gallery/: index.json con el listado y packs/*.json con cada pack. Se añaden por pull request. La app no los descarga de raw.githubusercontent: Trunk copia gallery/ dentro de dist/ durante el build y la UI la lee por ruta relativa (gallery/index.json). Así funciona con el repositorio privado, en GitHub Pages, en trunk serve y dentro del binario de escritorio, que no tiene red.

```
{
  "file": "packs/nas.json",
  "name": "NAS casero",
  "desc": "samba, nfs-kernel-server y smartmontools"
}
```

Un pack se aplica como capa aditiva sobre la receta actual: añade paquetes, servicios y ficheros, nunca la reemplaza. También se pueden importar packs por URL desde la UI.

6. App de escritorio

rust/tauri-app sirve el bundle ya compilado de la UI en una ventana nativa y expone el núcleo por IPC como punto de extensión. Funciona sin conexión: la galería viaja en el bundle y el ZIP se genera con una copia local de fflate.

```
cd rust/web-ui && trunk build --release            # bundle que consume Tauri
cd rust/tauri-app/src-tauri && cargo tauri build   # paquete de la plataforma
cd rust/web-ui && trunk serve                      # desarrollo (Tauri apunta a
:8080)
```

La UI se compila antes y en su propio paso: los hooks de Tauri corren desde rust/tauri-app, no desde src-tauri, así que una ruta relativa no resuelve. El workflow de release empaqueta en matriz Linux (deb, rpm, appimage) y Windows (msi, nsis), sube artefactos en cada ejecución y publica la Release solo con un tag v*. Los iconos se generan con tools/make_icons.py.

7. Backend

server-rs es el backend, en axum. Es opcional: sin él la app funciona entera salvo las funciones de comunidad.

Método y ruta	Para qué
POST /api/register · /api/login · /api/logout	Cuentas con scrypt (salt:hash)
GET /api/me	Usuario de la sesión (cookie df_session o Bearer)
GET /api/packs?query=	Galería con búsqueda FTS5
POST /api/packs · /api/rating	Publicar un pack y puntuar
GET /api/testdrive · /api/testdrive/available	Sesiones de Test Drive y disponibilidad de QEMU
POST /api/testdrive · DELETE /api/testdrive/{id}	Arrancar y parar una máquina
GET /vnc.html · WS /ws/{id}	noVNC y puente websocket al VNC de QEMU
GET /api/health	Estado del servicio (lo usa el HEALTHCHECK del contenedor)

Variables: PORT (8080), DF_DB_PATH (por defecto :memory:; en producción, un volumen) y QEMU_BIN. La imagen está en GHCR y la guía de despliegue, con el aviso sobre cookies entre dominios distintos, en docs/DESPLIEGUE.md.

8. Integración continua

Workflow	Cuándo	Qué valida
ci.yml	push y PR	Núcleo con clippy, backend con tests, UI WASM con Trunk + wasm-opt y la interfaz en un navegador real
deploy.yml	push a main	Build de la UI y publicación en GitHub Pages
release.yml	tags v* o manual	Empaquetado de escritorio en Linux y Windows; Release con tag
server-image.yml	main, tags o manual	Imagen del backend publicada en GHCR

Los tags siguen el esquema vX.Y.Z. Todas las cajas del workspace comparten la misma versión.

9. Desarrollo y pruebas

```
cd rust && cargo test -p forge-core          # núcleo
cd rust/server-rs && cargo test              # backend (8 tests de integración)
cd rust/web-ui && trunk serve                # UI con recarga
python tools/make_icons.py                   # iconos de app y favicon
python tools/make_manual.py                  # este manual
```

Para añadir una base nueva: entra en el catálogo del núcleo (forge-core), añade la familia con su gestor de paquetes, su motor y sus valores por defecto, y cubre el caso en los tests. La UI y el CLI la recogen automáticamente porque ambos leen el mismo catálogo.

10. Seguridad

- Las contraseñas viajan en claro dentro de las recetas generadas: kickstart, preseed y AutoYaST lo exigen así. Usa una contraseña provisional en cualquier proyecto que vayas a compartir o publicar.
- Los repositorios personalizados se firman con GPG real en la receta.
- Las salidas OCI pueden incluir SBOM (syft), escaneo de vulnerabilidades (trivy) y firma (cosign).
- El backend guarda las contraseñas con scrypt (N=16384, r=8, p=1) y abre la sesión con una cookie HttpOnly.

11. Glosario

Término	Significado
Receta	Conjunto de ficheros que describen cómo construir la imagen
Motor	Herramienta que construye la imagen (KIWI NG, live-build, mkarchiso, kickstart)
Plantilla	Punto de partida preconfigurado (escritorio, servidor, JeOS…)
Pack	Capa aditiva de paquetes, servicios y ficheros
Test Drive	Arranque de la imagen en el navegador mediante QEMU y noVNC
Salida	Formato del artefacto: ISO, QCOW2, OCI, WSL…